

FamilyOS — Event Model v0.1

1. Objetivo

Definir os eventos que alteram o estado do FamilyOS e quais ações, alertas, tarefas ou atualizações devem ser disparados automaticamente.

Enquanto o **Knowledge Model v0.1** responde “o que existe” no universo familiar (entidades, atributos, relacionamentos), o **Event Model v0.1** responde “**o que muda, quando muda, e o que deve acontecer em consequência**”. É o modelo que transforma o grafo estático em um sistema vivo, reativo e proativo.

2. Estrutura padrão de um evento

Todo evento deverá conter:

Campo	Descrição
Event ID	Identificador único do evento
Event Type	Tipo do evento (ex.: <code>DocumentExpiring</code>)
Date/Time	Momento em que o evento ocorreu ou foi detectado
Source	Origem do evento (upload manual, integração, IA, planilha importada, e-mail, WhatsApp)
Affected Entity	Entidade do Knowledge Model impactada (Documento, Imóvel, Despesa, etc.)
Related Person	Pessoa(s) da família relacionada(s) ao evento
Previous State	Estado da entidade antes do evento
New State	Estado da entidade após o evento
Priority	Nível de urgência (baixa, média, alta, crítica)
Evidence/Document	Documento ou comprovante que evidencia o evento

Responsible Person	Pessoa responsável por tratar/confirmar o evento
Automation Status	Se o evento foi tratado automaticamente, parcialmente, ou requer ação manual

3. Categorias de eventos

3.1 Documentos

- DocumentUploaded
- DocumentClassified
- DocumentUpdated
- DocumentReplaced
- DocumentExpiring
- DocumentExpired
- DocumentMissing

3.2 Pessoas

- PersonCreated
- PersonalDataUpdated
- FamilyRoleChanged
- DependentAdded

3.3 Financeiro

- ExpenseCreated
- ExpenseDue
- ExpensePaid
- ExpenseOverdue
- RevenueExpected
- RevenueReceived

- RevenueOverdue
- RecurringChargeDetected
- UnexpectedChargeDetected

3.4 Patrimônio e imóveis

- PropertyAdded
- OwnershipChanged
- LeaseStarted
- LeaseEnded
- RentExpected
- RentReceived
- TaxIssued
- TaxDue
- InsuranceExpiring
- MaintenanceRequired
- PropertySold

3.5 Saúde

- ExamReceived
- ExamUpdated
- AppointmentScheduled
- AppointmentCompleted
- MedicationStarted
- MedicationChanged
- MedicationEnded
- VaccinationDue
- MedicalFollowUpRequired

3.6 Viagens

- `TripCreated`
- `TravelerAdded`
- `PassportExpiring`
- `VisaExpiring`
- `ESTAExpiring`
- `TravelRequirementChanged`
- `TravelReadinessChecked`
- `TravelReadinessFailed`
- `TravelReadinessApproved`

3.7 Governança

- `TaskCreated`
 - `TaskAssigned`
 - `TaskCompleted`
 - `DeadlineApproaching`
 - `DeadlineMissed`
 - `FamilyDecisionRecorded`
 - `RuleCreated`
 - `RuleChanged`
-

4. Modelo Evento → Regra → Ação

Cada evento deve mostrar o que acontece depois.

Exemplo 1 — Documento vencendo

Evento: `DocumentExpiring`

Condição: Validade em 90 dias

Ações:

1. Criar alerta
2. Criar pendência
3. Definir responsável
4. Incluir no briefing
5. Repetir alerta em 60, 30 e 7 dias

Exemplo 2 — Viagem criada

Evento: `TripCreated`

Ações:

1. Identificar viajantes
2. Verificar passaportes
3. Verificar vistos, ESTA ou ETA
4. Verificar vacinas e seguro
5. Calcular Travel Readiness
6. Criar tarefas para itens pendentes

Exemplo 3 — Despesa vencendo

Evento: `ExpenseDue`

Ações:

1. Notificar responsável
2. Incluir no fluxo financeiro
3. Incluir no briefing diário
4. Se não houver confirmação, mudar para `ExpenseOverdue`

Exemplo 4 — Exame recebido

Evento: `ExamReceived`

Ações:

1. Relacionar à pessoa

2. Classificar tipo e data
 3. Armazenar documento
 4. Atualizar linha do tempo de saúde
 5. Verificar consulta ou retorno pendente
-

5. Validação com o SeixasOS

Os arquivos reais trazidos como referência (planilha financeira mensal e checklist de obrigações recorrentes) confirmam que o Knowledge Model está no caminho certo.

O controle atual em uso reúne pagamentos mensais, cartões, assinaturas, tributos, seguros, recebíveis, aluguéis, veículos e obrigações recorrentes. Isso demonstra que o sistema precisa representar não apenas uma **Despesa**, mas também **recorrência**, **vencimento**, **confirmação de pagamento**, **responsável** e **evidência** — ou seja, o Event Model é indispensável para que o Knowledge Model ganhe vida.

A planilha de despesas também servirá para validar as entidades financeiras e, posteriormente, testar a importação e reconciliação de dados reais no piloto.

6. Ajustes futuros identificados no Knowledge Model

Durante a validação com dados reais, cinco conceitos precisam ser incorporados ao Knowledge Model v0.2 para que o Event Model funcione corretamente:

1. **Pagamento/Transação** — separado da obrigação ou despesa; um evento (`ExpensePaid`) precisa apontar para um registro de pagamento concreto, não apenas mudar o status da despesa.
2. **Recorrência** — para contas mensais e anuais; necessário para que `ExpenseDue` / `RevenueExpected` sejam gerados automaticamente a partir de um padrão, sem recriação manual.
3. **Comprovante** — ligado ao pagamento (não apenas ao documento genérico), como evidência específica de quitação.
4. **Fornecedor/Beneficiário** — entidade para quem recebe o pagamento (concessionária, seguradora, prestador de serviço), hoje ausente no modelo.
5. **Status operacional** — padronizar estados como *previsto*, *emitido*, *pago*, *vencido*,

contestado, cancelado, aplicável a Despesa, Receita e Tributo, para que os eventos tenham transições de estado consistentes.

7. Próximos Passos

1. Incorporar os 5 ajustes da seção 6 ao Knowledge Model (v0.2).
2. Definir o schema técnico de eventos (payload JSON, fila/log de eventos) em `04_Architecture`.
3. Mapear quais eventos entram no MVP: `DocumentExpiring`, `DocumentExpired`, `ExpenseDue`, `ExpenseOverdue`, `TravelReadinessChecked` — alinhados ao escopo do Vision & Product Definition v0.1.
4. Testar o modelo Evento → Regra → Ação com dados reais do SeixasOS (planilha e checklist).